Validación cruzada y pipelines Python

Edison Achalma
Escuela Profesional de Economía, Universidad Nacional de San Cristóbal de Huamanga

Resumen

Este abstract será actualizado una vez que se complete el contenido final del artículo.

Tabla de contenidos

Introduction **3**

1 El problema del overfitting **3**
1.1 Fundamento teórico . . . 3
1.1.1 Descomposición del error de predicción . . . 3
1.1.2 Trade-off sesgo-varianza . . . 3
1.1.3 Señales de overfitting . . . 4
1.2 Ilustración del overfitting . . . 4
1.3 Visualización del trade-off . . . 6

2 Validación cruzada (Cross-Validation) **6**
2.1 Fundamento matemático . . . 6
2.1.1 Problema con validación simple . . . 7
2.1.2 K-Fold Cross-Validation . . . 7
2.1.3 Variantes de Cross-Validation . . . 8
2.2 Implementación básica . . . 8
2.3 Stratified Cross-Validation . . . 8
2.4 Cross-validation con múltiples métricas . . . 9
2.5 Visualización de resultados CV . . . 10

3 Grid Search y optimización de hiperparámetros **11**
3.1 Fundamento matemático . . . 11
3.1.1 Formulación del problema . . . 11
3.1.2 Grid Search (Búsqueda exhaustiva) . . . 11

Edison Achalma https://orcid.org/0000-0001-6996-3364

El autor no tiene conflictos de interés que revelar. Los roles de autor se clasificaron utilizando la taxonomía de roles de colaborador (CRediT; https://credit.niso.org/) de la siguiente manera: Edison Achalma: conceptualización, metodología, análisis formal, investigación, recursos, curación de datos, redacción, visualización, supervisión, administración del proyecto

La correspondencia relativa a este artículo debe dirigirse a Edison Achalma, Escuela Profesional de Economía, Universidad Nacional de San Cristóbal de Huamanga, Portal Independencia N° 57, Ayacucho, AYA 5001, Perú, Email: elmer.achalma.09@unsch.edu.pe

3.1.3 Random Search . . . 12
3.2 Implementación de Grid Search . . . 12
3.3 Análisis de resultados Grid Search . . . 13
3.4 Visualización de Grid Search . . . 14
3.5 Random Search implementation . . . 15

4 Curvas de aprendizaje **16**
4.1 Fundamento matemático . . . 16
4.1.1 Definición . . . 16
4.1.2 Comportamiento típico . . . 16
4.2 Implementación . . . 17
4.3 Comparación de modelos . . . 18

5 Contribución de la data al modelo **19**
5.1 Análisis de sensibilidad al tamaño de muestra . . . 19
5.2 Visualización . . . 20

6 Contribución de las variables **21**
6.1 Importancia de características . . . 21
6.2 Análisis de permutación . . . 21
6.3 Visualización de importancia . . . 22
6.4 Selección de características . . . 23
6.5 Análisis incremental de variables . . . 23

7 Aplicaciones económicas **25**
7.1 Aplicación 1: Predicción de incumplimiento crediticio . . . 25
7.2 Pipeline completo de optimización . . . 26

8 Mejores prácticas **28**
8.1 Checklist de validación de modelos . . . 28
8.2 Función de pipeline completo . . . 30

9 Conclusión **33**
9.1 Recursos adicionales . . . 34

10 Publicaciones Similares **34**

Validación cruzada y pipelines Python

En esta undécima y última guía exploraremos técnicas esenciales para evaluar y optimizar modelos de machine learning: validación cruzada, búsqueda de hiperparámetros, y análisis de contribución de datos y variables. Estas técnicas son fundamentales para desarrollar modelos robustos que generalicen bien a datos no vistos.

1 El problema del overfitting

1.1 Fundamento teórico

El **overfitting** (sobreajuste) ocurre cuando un modelo captura no solo los patrones verdaderos en los datos, sino también el ruido aleatorio. Este es uno de los problemas fundamentales en machine learning y econometría.

1.1.1 Descomposición del error de predicción

Para cualquier modelo de predicción, el error esperado fuera de muestra se descompone en tres componentes:

$$\mathbb{E}[(Y - \hat{f}(\mathbf{X}))^2] = \text{Bias}^2[\hat{f}(\mathbf{X})] + \text{Var}[\hat{f}(\mathbf{X})] + \sigma^2$$

donde:

1. Sesgo (Bias):

$$\text{Bias}[\hat{f}(\mathbf{X})] = \mathbb{E}[\hat{f}(\mathbf{X})] - f(\mathbf{X})$$

Mide qué tan lejos está la predicción promedio del valor verdadero. Modelos simples tienen alto sesgo (underfitting).

2. Varianza (Variance):

$$\text{Var}[\hat{f}(\mathbf{X})] = \mathbb{E}[(\hat{f}(\mathbf{X}) - \mathbb{E}[\hat{f}(\mathbf{X})])^2]$$

Mide cuánto varían las predicciones si se entrena con diferentes muestras. Modelos complejos tienen alta varianza (overfitting).

3. Error irreducible:

$$\sigma^2 = \text{Var}(\varepsilon)$$

Ruido inherente en los datos que ningún modelo puede reducir.

1.1.2 Trade-off sesgo-varianza

Aumentar la complejidad del modelo:

- **Reduce sesgo**: El modelo se ajusta mejor a los datos
- **Aumenta varianza**: El modelo se vuelve más sensible a cambios en los datos

Objetivo: Encontrar el nivel de complejidad que minimiza el error total.

1.1.3 Señales de overfitting

1. **Gap entre train y test**:

$$\text{Performance}_{\text{train}} >> \text{Performance}_{\text{test}}$$

2. **Alta complejidad del modelo**: Muchos parámetros relativos a observaciones
3. **Rendimiento perfecto en training**: Accuracy = 100%, $R^2 = 1$
4. **Inestabilidad**: Pequeños cambios en datos → grandes cambios en modelo

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Scikit-learn
from sklearn.datasets import load_iris
from sklearn.model_selection import (
    train_test_split, cross_val_score, cross_validate,
    GridSearchCV, RandomizedSearchCV, learning_curve,
    validation_curve, KFold, StratifiedKFold
)
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score,
    f1_score, roc_auc_score, roc_curve,
    confusion_matrix, classification_report
)

# Configuración
plt.style.use('seaborn-v0_8-whitegrid')
plt.rcParams['figure.figsize'] = (12, 6)
np.random.seed(42)

print("Librerías importadas exitosamente")
```

1.2 Ilustración del overfitting

```
# Cargar datos
iris = load_iris()
X = iris.data[:, :2]  # Solo 2 características para visualización
y = (iris.target == 2).astype(int)  # Clasificación binaria
```

```
print("ILUSTRACIÓN DE OVERFITTING")
print("=" * 70)

# Dividir datos
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)

# Entrenar modelos con diferente complejidad
modelos = {
    'Simple (k=50)': KNeighborsClassifier(n_neighbors=50),
    'Balanceado (k=5)': KNeighborsClassifier(n_neighbors=5),
    'Complejo (k=1)': KNeighborsClassifier(n_neighbors=1)
}

resultados = []

for nombre, modelo in modelos.items():
    modelo.fit(X_train, y_train)

    acc_train = modelo.score(X_train, y_train)
    acc_test = modelo.score(X_test, y_test)
    gap = acc_train - acc_test

    resultados.append({
        'Modelo': nombre,
        'Accuracy Train': acc_train,
        'Accuracy Test': acc_test,
        'Gap (Train-Test)': gap
    })

    print(f"\n{nombre}:")
    print(f"  Train: {acc_train:.4f}")
    print(f"  Test:  {acc_test:.4f}")
    print(f"  Gap:   {gap:.4f}")

    if gap > 0.15:
        print(f"    Posible overfitting detectado")
    elif acc_test < 0.70:
        print(f"    Posible underfitting detectado")
    else:
        print(f"   Balance adecuado")

df_resultados = pd.DataFrame(resultados)
```

1.3 Visualización del trade-off

```
# Evaluar KNN con diferentes k
k_values = range(1, 51)
train_scores = []
test_scores = []

for k in k_values:
    knn = KNeighborsClassifier(n_neighbors=k)
    knn.fit(X_train, y_train)

    train_scores.append(knn.score(X_train, y_train))
    test_scores.append(knn.score(X_test, y_test))

# Graficar
plt.figure(figsize=(12, 6))

plt.plot(k_values, train_scores, 'o-', label='Training', linewidth=2)
plt.plot(k_values, test_scores, 's-', label='Test', linewidth=2)

# Marcar mejor k
best_k = k_values[np.argmax(test_scores)]
best_score = max(test_scores)
plt.axvline(x=best_k, color='red', linestyle='--', alpha=0.7,
            label=f'Mejor k={best_k}')

plt.xlabel('k (número de vecinos)')
plt.ylabel('Accuracy')
plt.title('Trade-off Sesgo-Varianza en KNN')
plt.legend()
plt.grid(True, alpha=0.3)

# Anotar regiones
plt.text(5, 0.65, 'Alta varianza\n(Overfitting)',
         ha='center', fontsize=10, bbox=dict(boxstyle='round', facecolor='red', alpha
plt.text(40, 0.65, 'Alto sesgo\n(Underfitting)',
         ha='center', fontsize=10, bbox=dict(boxstyle='round', facecolor='blue', alph

plt.tight_layout()
plt.savefig('bias_variance_tradeoff.png', dpi=300, bbox_inches='tight')
print("\nGráfico guardado como 'bias_variance_tradeoff.png'")
```

2 Validación cruzada (Cross-Validation)

2.1 Fundamento matemático

La validación cruzada es una técnica para estimar el rendimiento de un modelo en datos no vistos mediante la división estratégica del conjunto de datos disponible.

2.1.1 Problema con validación simple

Con una sola división train-test:

- El resultado depende de qué observaciones caen en cada conjunto
- Alta varianza en la estimación del error
- Desperdicio de datos (test set no se usa para entrenar)

2.1.2 K-Fold Cross-Validation

Algoritmo:

1. Dividir datos en K folds (particiones) de tamaño aproximadamente igual
2. Para cada fold $k = 1, \ldots, K$:
   - Entrenar modelo en $K - 1$ folds
   - Validar en fold k
   - Calcular métrica de performance Score_k
3. Promedio de scores:
$$\text{CV Score} = \frac{1}{K} \sum_{k=1}^{K} \text{Score}_k$$
4. Desviación estándar:
$$\text{SD} = \sqrt{\frac{1}{K-1} \sum_{k=1}^{K} (\text{Score}_k - \overline{\text{Score}})^2}$$

Propiedades:

- **Sesgo**:
$$\mathbb{E}[\text{CV Score}] \approx \text{Error verdadero}$$
Estimación casi insesgada del error de generalización
- **Varianza**: Disminuye con K más grande
- **Trade-off K**:
  - K pequeño (ej: 5): Más sesgo, menos varianza, más rápido
  - K grande (ej: 10): Menos sesgo, más varianza, más lento
  - $K = n$ (LOOCV): Sesgo mínimo, varianza máxima, muy lento

2.1.3 Variantes de Cross-Validation

1. Stratified K-Fold:
Mantiene proporción de clases en cada fold:

$$\frac{n_k^{(c)}}{n_k} \approx \frac{n^{(c)}}{n} \quad \forall k, c$$

donde $n_k^{(c)}$ es número de observaciones de clase c en fold k.
Crucial para: Datasets desbalanceados
2. Leave-One-Out CV (LOOCV):
Caso especial donde $K = n$:

$$\text{LOOCV} = \frac{1}{n} \sum_{i=1}^{n} L(y_i, \hat{f}^{(-i)}(\mathbf{X}_i))$$

donde $\hat{f}^{(-i)}$ es modelo entrenado sin observación i.
Ventajas: Sesgo mínimo **Desventajas**: Computacionalmente costoso, alta varianza
3. Repeated K-Fold:
Repetir K-Fold con diferentes particiones aleatorias y promediar.

2.2 Implementación básica

```
print("\n\nVALIDACIÓN CRUZADA")
print("=" * 70)

# Modelo a evaluar
modelo = LogisticRegression(random_state=42)

# K-Fold CV estándar
kfold = KFold(n_splits=5, shuffle=True, random_state=42)

scores = cross_val_score(
    modelo, X, y,
    cv=kfold,
    scoring='accuracy'
)

print("\nK-Fold Cross-Validation (K=5):")
print(f"Scores por fold: {scores}")
print(f"Media: {scores.mean():.4f}")
print(f"Desviación estándar: {scores.std():.4f}")
print(f"Intervalo 95% confianza: [{scores.mean() - 1.96*scores.std():.4f}, "
      f"{scores.mean() + 1.96*scores.std():.4f}]")
```

2.3 Stratified Cross-Validation

```
# Verificar distribución de clases
print(f"\n\nDistribución de clases:")
print(f"Clase 0: {(y==0).sum()} ({(y==0).sum()/len(y)*100:.1f}%)")
print(f"Clase 1: {(y==1).sum()} ({(y==1).sum()/len(y)*100:.1f}%)")

# Stratified K-Fold
stratified_kfold = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

scores_stratified = cross_val_score(
    modelo, X, y,
    cv=stratified_kfold,
    scoring='accuracy'
)

print(f"\nStratified K-Fold CV:")
print(f"Media: {scores_stratified.mean():.4f}")
print(f"Desv. est: {scores_stratified.std():.4f}")

# Comparación
print(f"\nDiferencia con K-Fold estándar: "
      f"{scores_stratified.mean() - scores.mean():.4f}")
```

2.4 Cross-validation con múltiples métricas

```
# Evaluar múltiples métricas simultáneamente
scoring = {
    'accuracy': 'accuracy',
    'precision': 'precision',
    'recall': 'recall',
    'f1': 'f1',
    'roc_auc': 'roc_auc'
}

cv_results = cross_validate(
    modelo, X, y,
    cv=stratified_kfold,
    scoring=scoring,
    return_train_score=True
)

print("\n\nRESULTADOS CROSS-VALIDATION - MÚLTIPLES MÉTRICAS")
print("=" * 70)

for metric in ['accuracy', 'precision', 'recall', 'f1', 'roc_auc']:
    train_scores = cv_results[f'train_{metric}']
    test_scores = cv_results[f'test_{metric}']
```

```
    print(f"\n{metric.upper()}:")
    print(f"  Train: {train_scores.mean():.4f} ± {train_scores.std():.4f}")
    print(f"  Test:  {test_scores.mean():.4f} ± {test_scores.std():.4f}")
    print(f"  Gap:   {train_scores.mean() - test_scores.mean():.4f}")
```

2.5 Visualización de resultados CV

```
# Crear DataFrame para visualización
metrics_data = []

for metric in ['accuracy', 'precision', 'recall', 'f1', 'roc_auc']:
    for fold in range(5):
        metrics_data.append({
            'Métrica': metric,
            'Fold': fold + 1,
            'Train': cv_results[f'train_{metric}'][fold],
            'Test': cv_results[f'test_{metric}'][fold]
        })

df_metrics = pd.DataFrame(metrics_data)

# Graficar
fig, axes = plt.subplots(2, 3, figsize=(15, 10))
axes = axes.ravel()

for idx, metric in enumerate(['accuracy', 'precision', 'recall', 'f1', 'roc_auc']):
    data = df_metrics[df_metrics['Métrica'] == metric]

    x = np.arange(1, 6)
    axes[idx].plot(x, data['Train'].values, 'o-', label='Train', linewidth=2)
    axes[idx].plot(x, data['Test'].values, 's-', label='Test', linewidth=2)

    axes[idx].set_xlabel('Fold')
    axes[idx].set_ylabel('Score')
    axes[idx].set_title(f'{metric.upper()}')
    axes[idx].legend()
    axes[idx].grid(True, alpha=0.3)
    axes[idx].set_ylim([0, 1.05])

# Ocultar último subplot
axes[5].axis('off')

plt.tight_layout()
plt.savefig('cv_multiple_metrics.png', dpi=300, bbox_inches='tight')
print("\nGráfico guardado como 'cv_multiple_metrics.png'")
```

3 Grid Search y optimización de hiperparámetros

3.1 Fundamento matemático

La búsqueda de hiperparámetros óptimos es un problema de optimización sobre un espacio discreto de configuraciones.

3.1.1 Formulación del problema

Dado:

- Modelo $f(\mathbf{X}; \boldsymbol{\theta}, \boldsymbol{\lambda})$
  - $\boldsymbol{\theta}$: Parámetros (aprendidos de datos)
  - $\boldsymbol{\lambda}$: Hiperparámetros (especificados a priori)
- Espacio de hiperparámetros $\Lambda = \Lambda_1 \times \Lambda_2 \times \cdots \times \Lambda_m$
- Métrica de performance $S(\boldsymbol{\lambda})$ estimada por CV

Objetivo: Encontrar

$$\boldsymbol{\lambda}^* = \arg\max_{\boldsymbol{\lambda} \in \Lambda} S(\boldsymbol{\lambda})$$

3.1.2 Grid Search (Búsqueda exhaustiva)

Algoritmo:

1. Definir grilla de hiperparámetros:

$$\Lambda = \{\lambda_1^{(1)}, \ldots, \lambda_1^{(n_1)}\} \times \cdots \times \{\lambda_m^{(1)}, \ldots, \lambda_m^{(n_m)}\}$$

2. Para cada combinación $\boldsymbol{\lambda} \in \Lambda$:
   - Entrenar modelo con CV
   - Calcular $S(\boldsymbol{\lambda})$
3. Seleccionar $\boldsymbol{\lambda}^*$ con mejor S

Complejidad: $O(n_1 \times n_2 \times \cdots \times n_m \times K)$
donde K es número de folds.
Ventajas:

- Exhaustivo: Garantiza encontrar mejor combinación en grilla
- Paralelizable: Evaluaciones independientes

Desventajas:

- Costoso: Crece exponencialmente con número de hiperparámetros
- Solo explora puntos discretos

3.1.3 Random Search

En lugar de grilla, muestrear aleatoriamente del espacio de hiperparámetros:

$$\lambda^{(i)} \sim P(\lambda), \quad i = 1, \ldots, N$$

Ventajas sobre Grid Search:

- Más eficiente: Puede encontrar buenos valores con menos evaluaciones
- Explora mejor espacios continuos
- Menos sensible a hiperparámetros irrelevantes

Teorema (Bergstra & Bengio, 2012): Random Search converge al óptimo más rápido que Grid Search cuando pocos hiperparámetros importan.

3.2 Implementación de Grid Search

```
print("\n\nGRID SEARCH - OPTIMIZACIÓN DE HIPERPARÁMETROS")
print("=" * 70)

# Definir modelo y grilla de hiperparámetros
modelo = RandomForestClassifier(random_state=42)

param_grid = {
    'n_estimators': [50, 100, 200],
    'max_depth': [3, 5, 10, None],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 2, 4]
}

print(f"Espacio de búsqueda:")
print(f"  n_estimators: {param_grid['n_estimators']}")
print(f"  max_depth: {param_grid['max_depth']}")
print(f"  min_samples_split: {param_grid['min_samples_split']}")
print(f"  min_samples_leaf: {param_grid['min_samples_leaf']}")

n_combinations = (len(param_grid['n_estimators']) *
                  len(param_grid['max_depth']) *
                  len(param_grid['min_samples_split']) *
                  len(param_grid['min_samples_leaf']))

print(f"\nTotal de combinaciones: {n_combinations}")
print(f"Con CV de 5-fold: {n_combinations * 5} modelos a entrenar")
```

```
# Ejecutar Grid Search
grid_search = GridSearchCV(
    estimator=modelo,
    param_grid=param_grid,
```

```
    cv=StratifiedKFold(n_splits=5, shuffle=True, random_state=42),
    scoring='roc_auc',
    n_jobs=-1,  # Usar todos los cores
    verbose=1,
    return_train_score=True
)

print("\nEjecutando Grid Search...")
grid_search.fit(X, y)

print("\n\nRESULTADOS GRID SEARCH")
print("=" * 70)
print(f"\nMejores hiperparámetros:")
for param, value in grid_search.best_params_.items():
    print(f"  {param}: {value}")

print(f"\nMejor score (ROC-AUC): {grid_search.best_score_:.4f}")

# Evaluar mejor modelo
best_model = grid_search.best_estimator_
y_pred = best_model.predict(X)
y_proba = best_model.predict_proba(X)[:, 1]

print(f"\nPerformance del mejor modelo:")
print(f"  Accuracy: {accuracy_score(y, y_pred):.4f}")
print(f"  Precision: {precision_score(y, y_pred):.4f}")
print(f"  Recall: {recall_score(y, y_pred):.4f}")
print(f"  F1-Score: {f1_score(y, y_pred):.4f}")
print(f"  ROC-AUC: {roc_auc_score(y, y_proba):.4f}")
```

3.3 Análisis de resultados Grid Search

```
# Convertir resultados a DataFrame
results = pd.DataFrame(grid_search.cv_results_)

# Top 10 configuraciones
top_10 = results.nsmallest(10, 'rank_test_score')[
    ['params', 'mean_test_score', 'std_test_score',
     'mean_train_score', 'mean_fit_time']
]

print("\n\nTOP 10 CONFIGURACIONES")
print("=" * 70)
for idx, row in top_10.iterrows():
    print(f"\nRank {int(results.loc[idx, 'rank_test_score'])}:")
    print(f"  Parámetros: {row['params']}")
    print(f"  Test Score: {row['mean_test_score']:.4f} ± {row['std_test_score']:.4f}"
```

```
    print(f"  Train Score: {row['mean_train_score']:.4f}")
    print(f"  Tiempo: {row['mean_fit_time']:.2f}s")
```

3.4 Visualización de Grid Search

```
# Analizar efecto de hiperparámetros clave
fig, axes = plt.subplots(2, 2, figsize=(14, 10))

# 1. n_estimators vs score
param = 'param_n_estimators'
grouped = results.groupby(param).agg({
    'mean_test_score': 'mean',
    'std_test_score': 'mean'
}).reset_index()

axes[0, 0].errorbar(grouped[param], grouped['mean_test_score'],
                    yerr=grouped['std_test_score'], marker='o', capsize=5)
axes[0, 0].set_xlabel('n_estimators')
axes[0, 0].set_ylabel('ROC-AUC')
axes[0, 0].set_title('Efecto de n_estimators')
axes[0, 0].grid(True, alpha=0.3)

# 2. max_depth vs score
param = 'param_max_depth'
grouped = results.groupby(param).agg({
    'mean_test_score': 'mean',
    'std_test_score': 'mean'
}).reset_index()

# Reemplazar None con string para graficar
grouped[param] = grouped[param].astype(str)

axes[0, 1].errorbar(range(len(grouped)), grouped['mean_test_score'],
                    yerr=grouped['std_test_score'], marker='s', capsize=5)
axes[0, 1].set_xticks(range(len(grouped)))
axes[0, 1].set_xticklabels(grouped[param])
axes[0, 1].set_xlabel('max_depth')
axes[0, 1].set_ylabel('ROC-AUC')
axes[0, 1].set_title('Efecto de max_depth')
axes[0, 1].grid(True, alpha=0.3)

# 3. Heatmap: n_estimators vs max_depth
pivot_data = results.pivot_table(
    values='mean_test_score',
    index='param_max_depth',
    columns='param_n_estimators',
    aggfunc='mean'
```

```
)

sns.heatmap(pivot_data, annot=True, fmt='.3f', cmap='YlOrRd',
            ax=axes[1, 0], cbar_kws={'label': 'ROC-AUC'})
axes[1, 0].set_title('Heatmap: n_estimators vs max_depth')

# 4. Train vs Test scores (top 20)
top_20 = results.nsmallest(20, 'rank_test_score')

axes[1, 1].scatter(top_20['mean_train_score'], top_20['mean_test_score'],
                   alpha=0.6, s=100)
axes[1, 1].plot([0.8, 1.0], [0.8, 1.0], 'r--', label='Train=Test')
axes[1, 1].set_xlabel('Train Score')
axes[1, 1].set_ylabel('Test Score')
axes[1, 1].set_title('Train vs Test Scores (Top 20)')
axes[1, 1].legend()
axes[1, 1].grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig('grid_search_analysis.png', dpi=300, bbox_inches='tight')
print("\nGráfico guardado como 'grid_search_analysis.png'")
```

3.5 Random Search implementation

```
from sklearn.model_selection import RandomizedSearchCV
from scipy.stats import randint, uniform

print("\n\nRANDOM SEARCH")
print("=" * 70)

# Definir distribuciones para Random Search
param_distributions = {
    'n_estimators': randint(50, 300),
    'max_depth': [3, 5, 7, 10, 15, None],
    'min_samples_split': randint(2, 20),
    'min_samples_leaf': randint(1, 10),
    'max_features': ['sqrt', 'log2', None]
}

# Ejecutar Random Search
random_search = RandomizedSearchCV(
    estimator=RandomForestClassifier(random_state=42),
    param_distributions=param_distributions,
    n_iter=100,  # Número de combinaciones a probar
    cv=StratifiedKFold(n_splits=5, shuffle=True, random_state=42),
    scoring='roc_auc',
    n_jobs=-1,
```

```
    verbose=1,
    random_state=42,
    return_train_score=True
)

print("Ejecutando Random Search (100 iteraciones)...")
random_search.fit(X, y)

print(f"\nMejores hiperparámetros:")
for param, value in random_search.best_params_.items():
    print(f"  {param}: {value}")

print(f"\nMejor score: {random_search.best_score_:.4f}")

# Comparar con Grid Search
print(f"\nComparación:")
print(f"  Grid Search score:   {grid_search.best_score_:.4f}")
print(f"  Random Search score: {random_search.best_score_:.4f}")
print(f"  Diferencia:          {random_search.best_score_ - grid_search.best_score_:.
```

4 Curvas de aprendizaje

4.1 Fundamento matemático

Las curvas de aprendizaje muestran cómo el performance del modelo cambia con el tamaño del conjunto de entrenamiento.

4.1.1 Definición

Para tamaños de muestra $m \in \{m_1, m_2, \ldots, m_k\}$ donde $m_k = n$:
Error de entrenamiento:

$$\text{Error}_{\text{train}}(m) = \frac{1}{m} \sum_{i=1}^{m} L(y_i, \hat{f}_m(\mathbf{X}_i))$$

Error de validación:

$$\text{Error}_{\text{val}}(m) = \frac{1}{n-m} \sum_{i=m+1}^{n} L(y_i, \hat{f}_m(\mathbf{X}_i))$$

4.1.2 Comportamiento típico

High Bias (Underfitting):

- Error de train alto, se mantiene alto
- Error de validación alto, converge cerca de train
- Gap pequeño
- **Solución**: Modelo más complejo, más features

High Variance (Overfitting):

- Error de train bajo
- Error de validación alto, no converge
- Gap grande
- **Solución**: Más datos, regularización, modelo más simple

Good Fit:

- Errores convergen a valor razonable
- Gap pequeño

4.2 Implementación

```
print("\n\nCURVAS DE APRENDIZAJE")
print("=" * 70)

# Función para crear curvas de aprendizaje
def plot_learning_curves(modelo, X, y, title, filename):
    """
    Crea y visualiza curvas de aprendizaje
    """
    # Calcular curvas
    train_sizes, train_scores, val_scores = learning_curve(
        modelo, X, y,
        cv=5,
        n_jobs=-1,
        train_sizes=np.linspace(0.1, 1.0, 10),
        scoring='accuracy',
        random_state=42
    )

    # Calcular medias y desviaciones estándar
    train_mean = train_scores.mean(axis=1)
    train_std = train_scores.std(axis=1)
    val_mean = val_scores.mean(axis=1)
    val_std = val_scores.std(axis=1)

    # Graficar
    plt.figure(figsize=(10, 6))

    plt.plot(train_sizes, train_mean, 'o-', color='blue',
             label='Training score', linewidth=2)
    plt.fill_between(train_sizes, train_mean - train_std,
                     train_mean + train_std, alpha=0.1, color='blue')

    plt.plot(train_sizes, val_mean, 's-', color='orange',
             label='Cross-validation score', linewidth=2)
    plt.fill_between(train_sizes, val_mean - val_std,
                     val_mean + val_std, alpha=0.1, color='orange')
```

```
    plt.xlabel('Training Size')
    plt.ylabel('Accuracy')
    plt.title(title)
    plt.legend(loc='best')
    plt.grid(True, alpha=0.3)
    plt.ylim([0.5, 1.05])

    plt.tight_layout()
    plt.savefig(filename, dpi=300, bbox_inches='tight')
    print(f"\nGráfico guardado como '{filename}'")

    # Diagnóstico
    final_gap = train_mean[-1] - val_mean[-1]
    print(f"\nDiagnóstico para {title}:")
    print(f"  Final training score: {train_mean[-1]:.4f}")
    print(f"  Final validation score: {val_mean[-1]:.4f}")
    print(f"  Gap: {final_gap:.4f}")

    if final_gap > 0.15:
        print("    High variance (overfitting)")
        print("  Recomendaciones: Más datos, regularización, modelo más simple")
    elif val_mean[-1] < 0.70:
        print("    High bias (underfitting)")
        print("  Recomendaciones: Modelo más complejo, más features")
    else:
        print("   Balance adecuado")

    return train_sizes, train_mean, val_mean

# Modelos a comparar
modelos = {
    'Logistic Regression': LogisticRegression(random_state=42),
    'Decision Tree (depth=3)': DecisionTreeClassifier(max_depth=3, random_state=42),
    'Decision Tree (depth=20)': DecisionTreeClassifier(max_depth=20, random_state=42),
    'Random Forest': RandomForestClassifier(n_estimators=100, random_state=42)
}

# Crear curvas para cada modelo
for nombre, modelo in modelos.items():
    filename = f"learning_curve_{nombre.replace(' ', '_').lower()}.png"
    plot_learning_curves(modelo, X, y, f'Learning Curve: {nombre}', filename)
```

4.3 Comparación de modelos

```
# Comparar todas las curvas en un gráfico
fig, axes = plt.subplots(2, 2, figsize=(14, 10))
```

```
axes = axes.ravel()

for idx, (nombre, modelo) in enumerate(modelos.items()):
    # Calcular curvas
    train_sizes, train_scores, val_scores = learning_curve(
        modelo, X, y,
        cv=5,
        n_jobs=-1,
        train_sizes=np.linspace(0.1, 1.0, 10),
        scoring='accuracy',
        random_state=42
    )

    train_mean = train_scores.mean(axis=1)
    val_mean = val_scores.mean(axis=1)

    axes[idx].plot(train_sizes, train_mean, 'o-', label='Train', linewidth=2)
    axes[idx].plot(train_sizes, val_mean, 's-', label='Validation', linewidth=2)
    axes[idx].set_xlabel('Training Size')
    axes[idx].set_ylabel('Accuracy')
    axes[idx].set_title(nombre)
    axes[idx].legend()
    axes[idx].grid(True, alpha=0.3)
    axes[idx].set_ylim([0.5, 1.05])

plt.tight_layout()
plt.savefig('learning_curves_comparison.png', dpi=300, bbox_inches='tight')
print("\nComparación guardada como 'learning_curves_comparison.png'")
```

5 Contribución de la data al modelo

5.1 Análisis de sensibilidad al tamaño de muestra

```
print("\n\nCONTRIBUCIÓN DE LA DATA AL MODELO")
print("=" * 70)

# Analizar cómo performance cambia con tamaño de datos
sample_sizes = [0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9, 1.0]
modelo = RandomForestClassifier(n_estimators=100, random_state=42)

resultados_sizes = []

for size in sample_sizes:
    n_samples = int(len(X) * size)

    # Muestrear datos
    indices = np.random.choice(len(X), n_samples, replace=False)
```

```
    X_sample = X[indices]
    y_sample = y[indices]

    # Cross-validation
    scores = cross_val_score(modelo, X_sample, y_sample, cv=5, scoring='roc_auc')

    resultados_sizes.append({
        'Proporción': size,
        'N_samples': n_samples,
        'Mean_AUC': scores.mean(),
        'Std_AUC': scores.std()
    })

    print(f"\nProporción: {size:.1f} (n={n_samples})")
    print(f"  AUC: {scores.mean():.4f} ± {scores.std():.4f}")

df_sizes = pd.DataFrame(resultados_sizes)
```

5.2 Visualización

```
# Graficar relación tamaño-performance
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Performance vs tamaño
axes[0].errorbar(df_sizes['N_samples'], df_sizes['Mean_AUC'],
                 yerr=df_sizes['Std_AUC'], marker='o', capsize=5, linewidth=2)
axes[0].set_xlabel('Tamaño de muestra')
axes[0].set_ylabel('ROC-AUC')
axes[0].set_title('Performance vs Tamaño de Muestra')
axes[0].grid(True, alpha=0.3)

# Beneficio marginal
marginal_benefit = df_sizes['Mean_AUC'].diff()
axes[1].plot(df_sizes['N_samples'][1:], marginal_benefit[1:],
             'o-', linewidth=2)
axes[1].axhline(y=0, color='r', linestyle='--', alpha=0.5)
axes[1].set_xlabel('Tamaño de muestra')
axes[1].set_ylabel('Beneficio marginal (Δ AUC)')
axes[1].set_title('Beneficio Marginal de Más Datos')
axes[1].grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig('data_contribution.png', dpi=300, bbox_inches='tight')
print("\nGráfico guardado como 'data_contribution.png'")

# Análisis de rendimientos
print(f"\n\nANÁLISIS DE RENDIMIENTOS:")
```

```
print(f"Con 10% de datos (n={int(0.1*len(X))}): AUC = {df_sizes.loc[0, 'Mean_AUC']:.4
print(f"Con 50% de datos (n={int(0.5*len(X))}): AUC = {df_sizes.loc[4, 'Mean_AUC']:.4
print(f"Con 100% de datos (n={len(X)}): AUC = {df_sizes.loc[9, 'Mean_AUC']:.4f}")

improvement_10_50 = df_sizes.loc[4, 'Mean_AUC'] - df_sizes.loc[0, 'Mean_AUC']
improvement_50_100 = df_sizes.loc[9, 'Mean_AUC'] - df_sizes.loc[4, 'Mean_AUC']

print(f"\nMejora 10%→50%: {improvement_10_50:.4f}")
print(f"Mejora 50%→100%: {improvement_50_100:.4f}")

if improvement_50_100 < improvement_10_50 / 2:
    print("\n  Rendimientos decrecientes: Más datos tiene beneficio limitado")
else:
    print("\n  Más datos aún beneficioso")
```

6 Contribución de las variables

6.1 Importancia de características

```
print("\n\nCONTRIBUCIÓN DE LAS VARIABLES AL MODELO")
print("=" * 70)

# Usar todas las características de Iris
X_full = iris.data
y_full = (iris.target == 2).astype(int)

feature_names = iris.feature_names

# Entrenar modelo con todas las características
rf_full = RandomForestClassifier(n_estimators=200, random_state=42)
rf_full.fit(X_full, y_full)

# Obtener importancias
importances = rf_full.feature_importances_
indices = np.argsort(importances)[::-1]

print("\nImportancia de características (Random Forest):")
print("=" * 70)
for i, idx in enumerate(indices):
    print(f"{i+1}. {feature_names[idx]:<30} {importances[idx]:.4f}")
```

6.2 Análisis de permutación

```
from sklearn.inspection import permutation_importance

# Permutation importance (más robusto)
```

```
perm_importance = permutation_importance(
    rf_full, X_full, y_full,
    n_repeats=30,
    random_state=42,
    n_jobs=-1
)

# Comparar con importancia de Random Forest
print("\n\nCOMPARACIÓN DE MÉTODOS DE IMPORTANCIA")
print("=" * 70)
print(f"{'Característica':<30} {'RF Importance':<15} {'Perm Importance':<15}")
print("-" * 70)

for i in indices:
    print(f"{feature_names[i]:<30} "
          f"{importances[i]:<15.4f} "
          f"{perm_importance.importances_mean[i]:<15.4f}")
```

6.3 Visualización de importancia

```
# Graficar importancias
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Random Forest importance
axes[0].barh(range(len(indices)), importances[indices])
axes[0].set_yticks(range(len(indices)))
axes[0].set_yticklabels([feature_names[i] for i in indices])
axes[0].set_xlabel('Importancia')
axes[0].set_title('Feature Importance (Random Forest)')
axes[0].invert_yaxis()

# Permutation importance
perm_sorted_idx = perm_importance.importances_mean.argsort()[::-1]
axes[1].boxplot(
    [perm_importance.importances[i] for i in perm_sorted_idx],
    vert=False,
    labels=[feature_names[i] for i in perm_sorted_idx]
)
axes[1].set_xlabel('Decrease in Accuracy')
axes[1].set_title('Permutation Importance')

plt.tight_layout()
plt.savefig('feature_importance.png', dpi=300, bbox_inches='tight')
print("\nGráfico guardado como 'feature_importance.png'")
```

6.4 Selección de características

```
from sklearn.feature_selection import RFECV

print("\n\nSELECCIÓN DE CARACTERÍSTICAS")
print("=" * 70)

# Recursive Feature Elimination con CV
modelo_seleccion = LogisticRegression(random_state=42, max_iter=1000)

rfecv = RFECV(
    estimator=modelo_seleccion,
    step=1,
    cv=StratifiedKFold(5),
    scoring='roc_auc',
    n_jobs=-1
)

rfecv.fit(X_full, y_full)

print(f"\nNúmero óptimo de características: {rfecv.n_features_}")
print(f"\nCaracterísticas seleccionadas:")
for i, selected in enumerate(rfecv.support_):
    if selected:
        print(f"    {feature_names[i]}")
    else:
        print(f"    {feature_names[i]}")

# Graficar
plt.figure(figsize=(10, 6))
plt.plot(range(1, len(rfecv.cv_results_['mean_test_score']) + 1),
         rfecv.cv_results_['mean_test_score'], 'o-', linewidth=2)
plt.xlabel('Número de características')
plt.ylabel('ROC-AUC (CV)')
plt.title('Recursive Feature Elimination con Cross-Validation')
plt.grid(True, alpha=0.3)
plt.axvline(x=rfecv.n_features_, color='r', linestyle='--',
            label=f'Óptimo: {rfecv.n_features_} features')
plt.legend()
plt.tight_layout()
plt.savefig('rfecv_results.png', dpi=300, bbox_inches='tight')
print("\nGráfico guardado como 'rfecv_results.png'")
```

6.5 Análisis incremental de variables

```
# Evaluar contribución incremental de cada variable
print("\n\nANÁLISIS INCREMENTAL DE VARIABLES")
print("=" * 70)

# Ordenar por importancia
ordered_features = [indices[i] for i in range(len(indices))]

resultados_incremental = []

for n_features in range(1, len(ordered_features) + 1):
    features_subset = ordered_features[:n_features]
    X_subset = X_full[:, features_subset]

    # Cross-validation
    scores = cross_val_score(
        RandomForestClassifier(n_estimators=100, random_state=42),
        X_subset, y_full,
        cv=5,
        scoring='roc_auc'
    )

    resultados_incremental.append({
        'N_features': n_features,
        'Features': [feature_names[i] for i in features_subset],
        'Mean_AUC': scores.mean(),
        'Std_AUC': scores.std()
    })

    print(f"\n{n_features} feature(s): {feature_names[features_subset[-1]]}")
    print(f"  AUC: {scores.mean():.4f} ± {scores.std():.4f}")

df_incremental = pd.DataFrame(resultados_incremental)

# Graficar
plt.figure(figsize=(10, 6))
plt.errorbar(df_incremental['N_features'], df_incremental['Mean_AUC'],
             yerr=df_incremental['Std_AUC'], marker='o', capsize=5, linewidth=2)
plt.xlabel('Número de características (agregadas por importancia)')
plt.ylabel('ROC-AUC')
plt.title('Contribución Incremental de Variables')
plt.grid(True, alpha=0.3)
plt.xticks(range(1, len(feature_names) + 1))
plt.tight_layout()
plt.savefig('incremental_features.png', dpi=300, bbox_inches='tight')
print("\nGráfico guardado como 'incremental_features.png'")
```

7 Aplicaciones económicas

7.1 Aplicación 1: Predicción de incumplimiento crediticio

```
print("\n\n\nAPLICACIÓN: OPTIMIZACIÓN DE MODELO DE CRÉDITO")
print("=" * 70)

# Generar datos sintéticos de crédito
np.random.seed(42)
n_clientes = 1000

datos_credito = pd.DataFrame({
    'ingreso_mensual': np.random.lognormal(8, 0.5, n_clientes),
    'edad': np.random.normal(40, 12, n_clientes),
    'antiguedad_laboral': np.random.exponential(5, n_clientes),
    'ratio_deuda_ingreso': np.random.beta(2, 5, n_clientes),
    'num_creditos_anteriores': np.random.poisson(2, n_clientes),
    'score_crediticio': np.random.normal(650, 100, n_clientes),
    'tiene_hipoteca': np.random.choice([0, 1], n_clientes, p=[0.7, 0.3]),
    'tiene_ahorros': np.random.choice([0, 1], n_clientes, p=[0.4, 0.6])
})

# Variable dependiente (probabilidad de incumplimiento)
prob_incumplimiento = (
    -0.01 * datos_credito['ingreso_mensual'] / 1000 +
    -0.02 * datos_credito['edad'] +
    -0.03 * datos_credito['antiguedad_laboral'] +
    1.5 * datos_credito['ratio_deuda_ingreso'] +
    0.05 * datos_credito['num_creditos_anteriores'] +
    -0.005 * datos_credito['score_crediticio'] +
    -0.3 * datos_credito['tiene_hipoteca'] +
    -0.4 * datos_credito['tiene_ahorros'] +
    np.random.normal(0, 0.2, n_clientes)
)

# Normalizar a [0,1] y generar incumplimientos
from scipy.special import expit
prob_incumplimiento = expit(prob_incumplimiento)
datos_credito['incumplimiento'] = (np.random.random(n_clientes) < prob_incumplimiento

print(f"Dataset de crédito generado:")
print(f"  Total clientes: {n_clientes}")
print(f"  Tasa de incumplimiento: {datos_credito['incumplimiento'].mean():.2%}")

# Preparar datos
X_credito = datos_credito.drop('incumplimiento', axis=1)
y_credito = datos_credito['incumplimiento']
```

7.2 Pipeline completo de optimización

```
# 1. Comparar múltiples modelos con CV
modelos_credito = {
    'Logistic Regression': LogisticRegression(random_state=42, max_iter=1000),
    'Random Forest': RandomForestClassifier(random_state=42),
    'Decision Tree': DecisionTreeClassifier(random_state=42)
}

print("\n\nCOMPARACIÓN DE MODELOS")
print("=" * 70)

resultados_modelos = []

for nombre, modelo in modelos_credito.items():
    cv_scores = cross_validate(
        modelo, X_credito, y_credito,
        cv=5,
        scoring=['roc_auc', 'precision', 'recall'],
        return_train_score=True
    )

    resultados_modelos.append({
        'Modelo': nombre,
        'AUC_test': cv_scores['test_roc_auc'].mean(),
        'Precision': cv_scores['test_precision'].mean(),
        'Recall': cv_scores['test_recall'].mean()
    })

    print(f"\n{nombre}:")
    print(f"  AUC: {cv_scores['test_roc_auc'].mean():.4f}")
    print(f"  Precision: {cv_scores['test_precision'].mean():.4f}")
    print(f"  Recall: {cv_scores['test_recall'].mean():.4f}")

df_modelos = pd.DataFrame(resultados_modelos)

# 2. Optimizar mejor modelo (Random Forest)
print("\n\nOPTIMIZACIÓN DE HIPERPARÁMETROS")
print("=" * 70)

param_grid_credito = {
    'n_estimators': [100, 200, 300],
    'max_depth': [5, 10, 15, 20],
    'min_samples_split': [2, 5, 10],
    'class_weight': ['balanced', None]
}

grid_credito = GridSearchCV(
```

```
    RandomForestClassifier(random_state=42),
    param_grid_credito,
    cv=5,
    scoring='roc_auc',
    n_jobs=-1,
    verbose=0
)

print("Ejecutando Grid Search...")
grid_credito.fit(X_credito, y_credito)

print(f"\nMejores parámetros:")
for param, value in grid_credito.best_params_.items():
    print(f"  {param}: {value}")
print(f"\nMejor AUC: {grid_credito.best_score_:.4f}")

# 3. Análisis de importancia de variables
modelo_final = grid_credito.best_estimator_
importances_credito = modelo_final.feature_importances_

print(f"\n\nIMPORTANCIA DE VARIABLES")
print("=" * 70)
importances_df = pd.DataFrame({
    'Variable': X_credito.columns,
    'Importancia': importances_credito
}).sort_values('Importancia', ascending=False)

for _, row in importances_df.iterrows():
    bar = ' ' * int(row['Importancia'] * 50)
    print(f"{row['Variable']:<25} {row['Importancia']:.4f} {bar}")

# 4. Curvas de aprendizaje
print(f"\n\nCURVAS DE APRENDIZAJE")
print("=" * 70)

train_sizes, train_scores, val_scores = learning_curve(
    modelo_final, X_credito, y_credito,
    cv=5,
    n_jobs=-1,
    train_sizes=np.linspace(0.1, 1.0, 10),
    scoring='roc_auc'
)

plt.figure(figsize=(10, 6))
plt.plot(train_sizes, train_scores.mean(axis=1), 'o-', label='Train', linewidth=2)
plt.plot(train_sizes, val_scores.mean(axis=1), 's-', label='Validation', linewidth=2)
plt.xlabel('Tamaño de muestra')
```

```
plt.ylabel('ROC-AUC')
plt.title('Curvas de Aprendizaje - Modelo de Crédito Optimizado')
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig('learning_curve_credito.png', dpi=300, bbox_inches='tight')
print("Curva guardada como 'learning_curve_credito.png'")

# Diagnóstico final
gap_final = train_scores.mean(axis=1)[-1] - val_scores.mean(axis=1)[-1]
print(f"\nDiagnóstico:")
print(f"  Training AUC: {train_scores.mean(axis=1)[-1]:.4f}")
print(f"  Validation AUC: {val_scores.mean(axis=1)[-1]:.4f}")
print(f"  Gap: {gap_final:.4f}")

if gap_final < 0.05:
    print("    Modelo bien balanceado")
elif gap_final < 0.15:
    print("    Ligero overfitting")
else:
    print("    Overfitting significativo")
```

8 Mejores prácticas

8.1 Checklist de validación de modelos

```
print("\n\n\nCHECKLIST DE VALIDACIÓN DE MODELOS")
print("=" * 70)

checklist = """
 DIVISIÓN DE DATOS
   Separar train/test antes de cualquier análisis
   Usar stratified split para clases desbalanceadas
   Mantener test set intocado hasta evaluación final

 VALIDACIÓN CRUZADA
   Usar K-Fold (típicamente K=5 o K=10)
   Stratified K-Fold para clasificación
   Repetir con diferentes seeds
   Reportar media ± desviación estándar

 OPTIMIZACIÓN DE HIPERPARÁMETROS
   Definir grilla razonable de hiperparámetros
   Usar validación cruzada anidada
   Considerar Random Search para espacios grandes
   Guardar mejores parámetros
```

```
 EVALUACIÓN DE OVERFITTING
   Comparar performance train vs test
   Analizar curvas de aprendizaje
   Verificar gap < 0.15
   Usar regularización si es necesario

 ANÁLISIS DE DATOS
   Evaluar si más datos ayudarían
   Verificar rendimientos decrecientes
   Considerar costo de recolección

 SELECCIÓN DE VARIABLES
   Analizar importancia de características
   Considerar RFECV para reducir dimensionalidad
   Evaluar contribución incremental
   Remover variables redundantes

 MÉTRICAS APROPIADAS
   Elegir métrica según objetivo de negocio
   Considerar costos de errores (FP vs FN)
   Reportar múltiples métricas
   Usar curvas ROC/PR para panorama completo

 ROBUSTEZ
   Probar con diferentes particiones
   Validar en período temporal diferente
   Verificar estabilidad con diferentes seeds
   Detectar data leakage

 DOCUMENTACIÓN
   Guardar hiperparámetros óptimos
   Documentar proceso de selección
   Registrar performance en test
   Explicar decisiones de diseño

 COMPARACIÓN
   Establecer baseline simple
   Comparar múltiples algoritmos
   Usar mismos datos para comparación
   Considerar trade-offs (interpretabilidad vs accuracy)
"""

print(checklist)
```

8.2 Función de pipeline completo

```
def pipeline_validacion_completo(X, y, modelos, param_grids, feature_names=None):
    """
    Pipeline completo de validación y optimización de modelos

    Parameters:
    -----------
    X : array-like
        Características
    y : array-like
        Variable objetivo
    modelos : dict
        Diccionario con modelos a evaluar
    param_grids : dict
        Diccionario con grillas de hiperparámetros
    feature_names : list, optional
        Nombres de características

    Returns:
    --------
    dict
        Resultados completos del pipeline
    """

    print("="*70)
    print("PIPELINE DE VALIDACIÓN COMPLETO")
    print("="*70)

    resultados = {
        'comparacion_modelos': [],
        'mejores_modelos': {},
        'importancias': {},
        'diagnostico': {}
    }

    # 1. Comparación inicial de modelos
    print("\n1. COMPARACIÓN INICIAL DE MODELOS")
    print("-"*70)

    for nombre, modelo in modelos.items():
        print(f"\nEvaluando {nombre}...")

        cv_results = cross_validate(
            modelo, X, y,
            cv=StratifiedKFold(5, shuffle=True, random_state=42),
            scoring=['roc_auc', 'precision', 'recall', 'f1'],
            return_train_score=True,
```

```
            n_jobs=-1
        )

        resultados['comparacion_modelos'].append({
            'Modelo': nombre,
            'AUC_train': cv_results['train_roc_auc'].mean(),
            'AUC_test': cv_results['test_roc_auc'].mean(),
            'AUC_std': cv_results['test_roc_auc'].std(),
            'Precision': cv_results['test_precision'].mean(),
            'Recall': cv_results['test_recall'].mean(),
            'F1': cv_results['test_f1'].mean(),
            'Gap': cv_results['train_roc_auc'].mean() - cv_results['test_roc_auc'].me
        })

        print(f"  AUC: {cv_results['test_roc_auc'].mean():.4f} ± "
              f"{cv_results['test_roc_auc'].std():.4f}")
        print(f"  Gap: {cv_results['train_roc_auc'].mean() - cv_results['test_roc_auc

    df_comparacion = pd.DataFrame(resultados['comparacion_modelos'])

    # 2. Optimización de hiperparámetros
    print("\n\n2. OPTIMIZACIÓN DE HIPERPARÁMETROS")
    print("-"*70)

    for nombre in modelos.keys():
        if nombre in param_grids:
            print(f"\nOptimizando {nombre}...")

            grid = GridSearchCV(
                modelos[nombre],
                param_grids[nombre],
                cv=5,
                scoring='roc_auc',
                n_jobs=-1,
                verbose=0
            )

            grid.fit(X, y)

            resultados['mejores_modelos'][nombre] = {
                'modelo': grid.best_estimator_,
                'params': grid.best_params_,
                'score': grid.best_score_
            }

            print(f"  Mejor AUC: {grid.best_score_:.4f}")
            print(f"  Mejores parámetros: {grid.best_params_}")
```

```
# 3. Análisis de importancia (para modelos con feature_importances_)
print("\n\n3. ANÁLISIS DE IMPORTANCIA DE VARIABLES")
print("-"*70)

for nombre, info in resultados['mejores_modelos'].items():
    modelo = info['modelo']

    if hasattr(modelo, 'feature_importances_'):
        importances = modelo.feature_importances_
        resultados['importancias'][nombre] = importances

        print(f"\n{nombre}:")
        if feature_names is not None:
            for i in np.argsort(importances)[::-1]:
                print(f"  {feature_names[i]:<30} {importances[i]:.4f}")
        else:
            for i, imp in enumerate(importances):
                print(f"  Feature {i}: {imp:.4f}")

# 4. Diagnóstico final
print("\n\n4. DIAGNÓSTICO Y RECOMENDACIONES")
print("-"*70)

mejor_modelo_nombre = df_comparacion.loc[df_comparacion['AUC_test'].idxmax(), 'Mo
mejor_auc = df_comparacion['AUC_test'].max()

resultados['diagnostico']['mejor_modelo'] = mejor_modelo_nombre
resultados['diagnostico']['mejor_auc'] = mejor_auc

print(f"\nMejor modelo: {mejor_modelo_nombre}")
print(f"AUC: {mejor_auc:.4f}")

# Verificar overfitting
gap = df_comparacion.loc[df_comparacion['Modelo'] == mejor_modelo_nombre, 'Gap'].

if gap > 0.15:
    print("\n  OVERFITTING DETECTADO")
    print("Recomendaciones:")
    print("  - Recolectar más datos")
    print("  - Aumentar regularización")
    print("  - Simplificar modelo")
    print("  - Remover variables poco importantes")
elif gap > 0.10:
    print("\n  Ligero overfitting")
    print("Recomendaciones:")
    print("  - Considerar regularización adicional")
    print("  - Validar en conjunto temporal diferente")
```

```
    else:
        print("\n  Modelo bien balanceado")

    return resultados

# Ejemplo de uso
print("\n\nEJEMPLO DE PIPELINE COMPLETO")
print("=" * 70)

modelos_ejemplo = {
    'Logistic Regression': LogisticRegression(random_state=42, max_iter=1000),
    'Random Forest': RandomForestClassifier(random_state=42),
    'Decision Tree': DecisionTreeClassifier(random_state=42)
}

param_grids_ejemplo = {
    'Random Forest': {
        'n_estimators': [100, 200],
        'max_depth': [5, 10, None],
        'min_samples_split': [2, 5]
    },
    'Decision Tree': {
        'max_depth': [3, 5, 7, 10],
        'min_samples_split': [2, 5, 10]
    }
}

resultados_pipeline = pipeline_validacion_completo(
    X_credito, y_credito,
    modelos_ejemplo,
    param_grids_ejemplo,
    feature_names=X_credito.columns.tolist()
)
```

9 Conclusión

La validación cruzada y optimización de modelos son fundamentales para desarrollar sistemas de machine learning robustos y confiables. La clave está en:

1. **No confiar en una sola métrica o división**
2. **Usar validación cruzada sistemáticamente**
3. **Optimizar hiperparámetros con datos de validación**
4. **Diagnosticar y corregir overfitting**
5. **Analizar contribución de datos y variables**
6. **Documentar decisiones y resultados**

Con estas técnicas, los economistas pueden desarrollar modelos predictivos que generalicen bien a nuevos datos y proporcionen insights valiosos para la toma de decisiones.

9.1 Recursos adicionales

Para profundizar en validación y optimización:

- Hastie, Tibshirani & Friedman - “Elements of Statistical Learning”
- James et al. - “Introduction to Statistical Learning”
- Scikit-learn Model Selection: scikit-learn.org/stable/model_selection.html
- Cross-validation strategies: scikit-learn.org/stable/modules/cross_validation.html
- Hyperparameter tuning guide: scikit-learn.org/stable/modules/grid_search.html

---

tipo: fragmento titulo: texto de apertura del bloque «Publicaciones similares» (incluido en cada post) —

10 Publicaciones Similares

Si te interesó este artículo, te recomendamos que explores otros blogs y recursos relacionados que pueden ampliar tus conocimientos. Aquí te dejo algunas sugerencias:

1. Instalacion De Anaconda
2. Configurar Entorno Virtual Python Anaconda
3. 01 Introducion A La Programacion Con Python
4. 02 Variables Expresiones Y Statements Con Python
5. 03 Objetos De Python
6. 04 Ejecucion Condicional Con Python
7. 05 Iteraciones Con Python
8. 06 Funciones Con Python
9. 07 Dataframes Con Python
10. 08 Prediccion Y Metrica De Performance Con Python
11. 09 Metodos De Machine Learning Para Clasificacion Con Python
12. 10 Metodos De Machine Learning Para Regresion Con Python
13. 11 Validacion Cruzada Y Composicion Del Modelo Con Python
14. Visualizacion De Datos Con Python

---

tipo: fragmento titulo: texto de cierre del bloque «Publicaciones similares» (incluido en cada post) —

Esperamos que encuentres estas publicaciones igualmente interesantes y útiles. ¡Disfruta de la lectura!